

# CS300 - CSC13002: Introduction to Software Engineering

---

## Project Assignment 1 (PA1-2026) Report

---

### Table of Contents

- [A - Group Registration](#)
  - [Group Information](#)
  - [Team Members List](#)
- [B - Project Proposal](#)
  - [B.1. Introduction](#)
  - [B.2. Target Users and Environments](#)
    - [Target Users](#)
    - [Environments](#)
  - [B.3. Key Features](#)
    - [Feature 1: Authentication and Security](#)
    - [Feature 2: Profile Management](#)
    - [Feature 3: Wallet and Personal Account Management](#)
    - [Feature 4: Personal Transaction Tracking](#)
    - [Feature 5: Group Management and Shared Wallets](#)
    - [Feature 6: Smart Debt Split and Settlement](#)
    - [Feature 7: Budget Planning and Alerting](#)
    - [Feature 8: Financial Reporting and Analytics](#)
    - [Feature 9: Savings Goal Management](#)
    - [Feature 10: System Administration \(Admin Panel\)](#)
  - [B.4. AI Feature](#)
    - [AI Financial Advisor](#)
- [C - Existing App Survey](#)
  - [C.1. App 1: Money Lover](#)
    - [Overview](#)
    - [Screenshots and Analysis](#)
    - [Key Strengths](#)
    - [Key Limitations](#)
  - [C.2. App 2: Splitwise](#)
    - [Overview](#)
    - [Screenshots and Analysis](#)
    - [Key Strengths](#)
    - [Key Limitations](#)
  - [C.3. Comparison and Differentiation](#)
    - [Feature Comparison Table](#)
    - [What Our App Will Do Differently or Better](#)
    - [UI/UX Patterns We Plan to Adopt](#)
- [D - Team Contract](#)
  - [D.1. Team Roles and Responsibilities](#)
  - [D.2. Communication Plan](#)

- Communication Tools
    - Meeting Schedule
    - Communication Protocols
  - D.3. Work Schedule and Deadlines
    - Project Milestones
    - Availability and Work Sessions
    - Contingency Plans
  - D.4. Code and Documentation Standards
    - Coding Conventions
    - Code Review Process
    - Documentation Standards
  - D.5. Accountability and Performance
    - Contribution Measurement
    - Handling Underperformance
    - Consequences
  - D.6. Decision-Making Process
  - D.7. Conflict Resolution
    - Resolution Framework
    - Ground Rules
  - D.8. Review and Update Process
- E - Development Tools and Process Setup
    - E.1. Development Process
    - E.2. Communication Tools
    - E.3. Task Management with Jira
    - E.4. Version Control and Repository (GitHub)
    - E.5. AI-Assisted Development Tools
    - E.6. Spec Kit
    - E.7. Tool Setup Summary

## A - Group Registration

Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Nguyễn Lê Đức Nhật

### Group Information

- **Group Name:** Group 04
- **Project Name:** FinSync
- **Date:** September 2026

### Team Members List

| # | Student ID | Full Name           | Email                     | Role         |
|---|------------|---------------------|---------------------------|--------------|
| 1 | 24120087   | Phạm Định Tiểu Long | phamlongkh2006@gmail.com  | Group Leader |
| 2 | 24120403   | Nguyễn Lê Đức Nhật  | nldnhat182006@gmail.com   | Member       |
| 3 | 24120051   | Ngô Thái Hòa        | ngothaihoa235@gmail.com   | Member       |
| 4 | 24120342   | Vương Đắc Gia Khiêm | vuongkiemvl10@gmail.com   | Member       |
| 5 | 24120038   | Nguyễn Phú Đạt      | nguyennphuudatt@gmail.com | Member       |

## B - Project Proposal

Performed by: Nguyễn Lê Đức Nhật & Ngô Thái Hòa (AI Feature) | Reviewed by: Phạm Định Tiểu Long | Edited by: Nguyễn Lê Đức Nhật

### B.1. Introduction

Performed by: Nguyễn Lê Đức Nhật | Reviewed by: Phạm Định Tiểu Long | Edited by: Nguyễn Lê Đức Nhật

FinSync is a comprehensive financial management application that combines personal asset management (similar to Money Lover) with intelligent group expense splitting (similar to Splitwise). The app is designed for individuals, students, and groups of users who need to track shared expenses such as room rent, travel trips, and events.

Current personal finance apps lack group expense management capabilities, while group splitting apps do not offer personal budgeting and account features. FinSync solves this problem by unifying personal and group finance into a single ecosystem, enhanced by an AI-powered financial advisor that analyzes spending history and recommends optimal spending plans for the upcoming month.

The system acts purely as a bookkeeping and reminder tool — it does **not** automatically deduct money or interact with real bank accounts.

### B.2. Target Users and Environments

Performed by: Nguyễn Lê Đức Nhật | Reviewed by: Phạm Định Tiểu Long | Edited by: Nguyễn Lê Đức Nhật

#### Target Users

##### 1. Regular User:

Individual users (students, young professionals) who need to track their personal cash flow, set savings goals, and simultaneously create or join group spending pools to share bills with friends or colleagues. Within a group, a user can hold one of two roles:

- **Owner:** The person who creates and manages the group.
- **Member:** A participant who joins an existing group.

##### 2. Administrator:

A system administrator who accesses a simple web-based admin panel to perform basic administrative tasks: viewing user lists, locking/unlocking accounts, managing default income/expense categories, and viewing aggregate statistics (total users, total transactions).

#### Environments

| Component                       | Technology               | Platform     |
|---------------------------------|--------------------------|--------------|
| Mobile App (Regular User)       | Kotlin (Jetpack Compose) | Android      |
| AI Microservice                 | Python (FastAPI)         | Cloud Server |
| Admin Web Panel (Administrator) | React.js                 | Web Browser  |
| Backend API Server              | Spring Boot (Java)       | Cloud Server |
| Database                        | PostgreSQL               | Cloud Server |

The mobile application targets Android devices and is built with Kotlin and Jetpack Compose. The AI microservice is built with Python (FastAPI) to handle expense analytics algorithms and AI integration. The admin panel is a simple web application built with React.js. All client and microservice components connect to a shared Spring Boot backend via RESTful APIs.

## B.3. Key Features

*Performed by: Nguyễn Lê Đức Nhật | Reviewed by: Phạm Đình Tiểu Long | Edited by: Nguyễn Lê Đức Nhật*

### Feature 1: Authentication and Security

The authentication module handles user registration, login, logout, and password recovery using encrypted JWT tokens. It ensures that all personal and group financial data remains private and secure, with only authorized accounts able to access their respective information.

### Feature 2: Profile Management

This feature allows users to view and edit their personal information, change their password, update their profile picture, and configure general settings such as the default currency (VND, USD). It provides a personalized experience and ensures user preferences are consistently applied throughout the app.

### Feature 3: Wallet and Personal Account Management

Users can create and manage bookkeeping wallets (cash, bank account, credit card) to track their balances. These wallets are purely manual-entry — they do not link to or affect real bank accounts. The system aggregates all wallet balances into a single dashboard providing an overview of the user's net worth.

### Feature 4: Personal Transaction Tracking

Users can quickly record personal income and expense entries by specifying the amount, category, notes, date, and an optional receipt image. This maintains an accurate personal ledger over time and allows users to review their spending patterns at a glance.

### Feature 5: Group Management and Shared Wallets

Users can create groups (for shared housing, travel, projects, etc.), invite members via a link or code, and assign roles (Owner and Member). Each group has its own shared wallet or ledger that is completely separated from personal finances, ensuring clarity between individual and collective spending.

### Feature 6: Smart Debt Split and Settlement

This feature automatically splits group expenses using flexible methods: equal split, percentage-based split, or exact-amount split. The system displays a consolidated summary of who owes whom and how much, sends payment reminder notifications to members with outstanding debts, and allows users to mark debts as "settled" once payment is confirmed outside the app.

### Feature 7: Budget Planning and Alerting

Users can set spending limits for individual personal categories or group funds on a weekly or monthly basis. The system triggers visual alerts when spending approaches or exceeds defined thresholds (at 80% and 100%), helping users stay within their financial boundaries.

### Feature 8: Financial Reporting and Analytics

This feature visualizes cash flow structure through interactive charts (pie charts, bar charts). For groups, the system generates expense summary reports after trips or on a monthly basis. Reports can be exported as PDF or Excel files for record-keeping and sharing.

### Feature 9: Savings Goal Management

Users can set long-term financial goals (e.g., buying a laptop, funding a trip), track their progress toward each goal, and manually or automatically allocate surplus funds. This encourages disciplined saving behavior and gives users a clear roadmap to their financial targets.

Feature 10: System Administration (Admin Panel)

The administrator can manage users (view list, lock/unlock accounts), manage default system income/expense categories, and view aggregate platform statistics. This ensures the platform remains well-maintained and provides oversight for user management.

B.4. AI Feature

Performed by: Ngô Thái Hòa | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Ngô Thái Hòa

AI Financial Advisor

Description:

The AI Financial Advisor is an intelligent spending consultant integrated into the app. At the beginning of each month (or on-demand), the AI automatically scans the user's entire transaction history from the previous month — including personal income, personal expenses, and group spending — to analyze cash flow structure, detect abnormal or spiking expense categories, and propose a practical spending plan for the current month.

How it works:

- Comparison Analysis:** The AI compares actual spending in each category against the user's set budget and the average of previous months.
- Anomaly Detection:** It flags categories with spending trends that exceed the budget or spike abnormally (e.g.,  
*"Your dining expenses last month increased by 40% compared to the 3-month average. Consider limiting it to 2.5 million VND this month."*  
).
- Optimized Budget Allocation:** It suggests an optimal budget allocation for each category based on the user's income, savings goals, and actual spending habits.
- Savings Progress Evaluation:** It evaluates progress toward savings goals and provides adjustment recommendations if the user is falling behind.

**Technology:** The backend calls an AI model API (Google Gemini API / OpenAI API) to analyze spending data and generate financial recommendations.

User Value:

This feature transforms raw spending data into proactive financial guidance. Users not only know what they have spent but also receive actionable advice on how they should spend in the coming month, fostering healthy financial habits and helping achieve long-term savings goals.

C - Existing App Survey

Performed by: Ngô Thái Hòa (Money Lover) & Vương Đức Gia Khiêm (Splitwise) | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Ngô Thái Hòa & Vương Đức Gia Khiêm

C.1. App 1: Money Lover

Performed by: Ngô Thái Hòa | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Ngô Thái Hòa

**App Name:** Money Lover  
**Platform:** Android, iOS, Web  
**Website:** <https://moneylover.me>

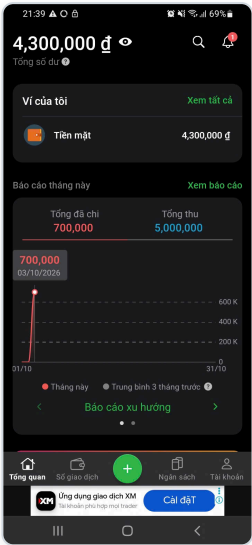
Overview

Money Lover is one of the most popular personal finance management apps in Vietnam and Southeast Asia. It allows users to track daily income and expenses, manage multiple wallets, set budgets, and view financial reports

through charts.

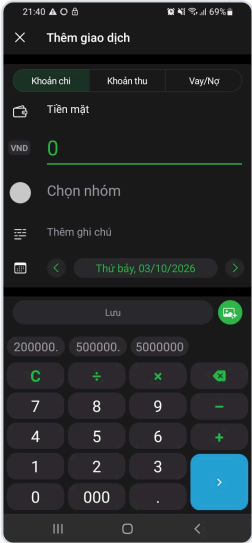
Screenshots and Analysis

Screen 1: Dashboard / Home Screen



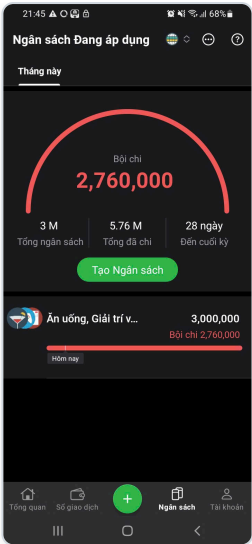
Caption: The home screen shows the user's wallet balance, recent transactions, and a summary of income/expenses for the current period. The clean layout provides quick access to add new transactions.

Screen 2: Transaction Entry



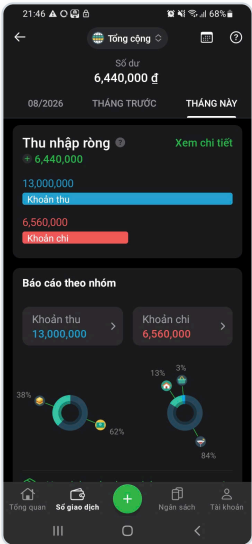
Caption: The transaction entry screen allows users to select a category, enter an amount, add notes, choose a wallet, and set a date. Categories are displayed as icons for quick selection.

Screen 3: Budget Management



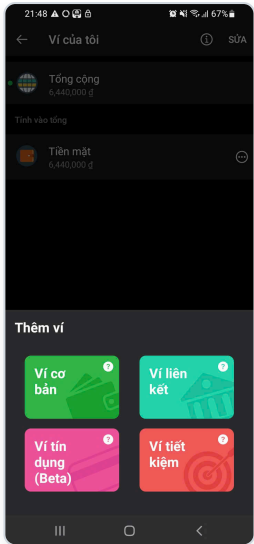
Caption: The budget screen shows spending progress bars for each category. Users can set monthly limits and receive warnings when approaching the threshold.

Screen 4: Reports and Charts



Caption: The report section offers pie charts and bar charts showing expense distribution by category and income vs. expense trends over time.

Screen 5: Wallet Management



Caption: Users can create and manage multiple wallets (cash, bank, credit card) with individual balances that aggregate into a total net worth view.

Key Strengths

- Intuitive and clean user interface
- Rich category system with custom categories
- Multi-wallet support with balance tracking
- Detailed financial reports with interactive charts
- Budget setting and tracking with alerts

Key Limitations

- No group expense management or bill splitting features
- No AI-powered spending analysis or recommendations
- Limited to personal finance only

C.2. App 2: Splitwise

Performed by: Vương Đức Gia Khiêm | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Vương Đức Gia Khiêm

**App Name:** Splitwise  
**Platform:** Android, iOS, Web  
**Website:** <https://www.splitwise.com>

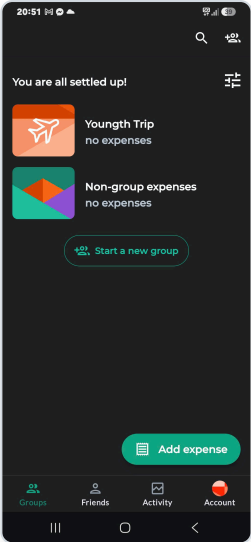
Overview

Splitwise is the leading group expense management app worldwide. It simplifies splitting bills among friends, roommates, and travel companions by tracking who owes whom and minimizing the number of transactions needed to settle all debts.

Screenshots and Analysis

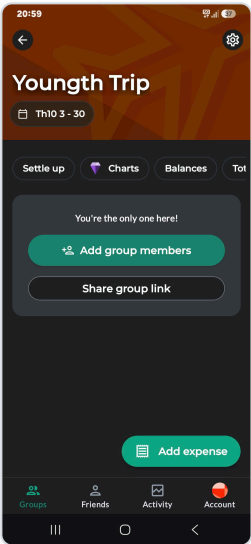
Screen 1: Group List / Dashboard





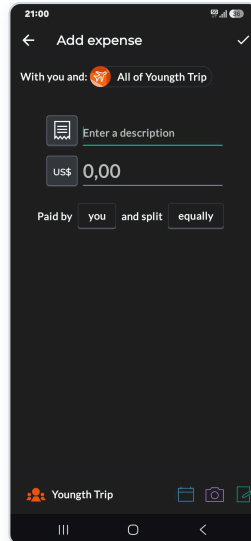
Caption: The main screen displays all groups the user belongs to, showing the total balance (amount owed or owed to the user) for each group. Color coding (green for positive, red for negative) provides quick visual feedback.

Screen 2: Group Detail and Expenses



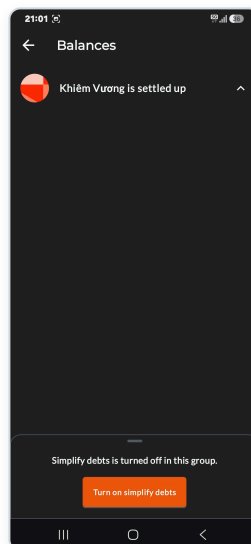
Caption: Inside a group, all shared expenses are listed chronologically with the payer and amount. Each expense shows how it was split among members.

Screen 3: Add Expense



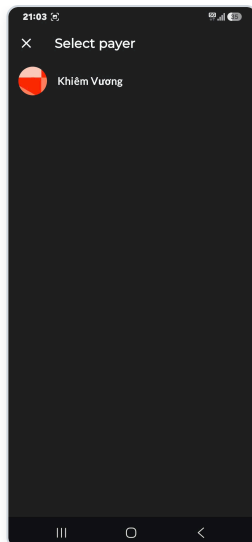
*Caption: When adding a group expense, the user specifies who paid, the total amount, and how to split it (equally, by exact amounts, or by percentages). The UI makes it easy to include or exclude specific members.*

#### Screen 4: Balance Summary



*Caption: The balance tab shows a simplified debt summary — who owes whom and how much — using a debt simplification algorithm that minimizes the number of transfers needed.*

#### Screen 5: Settle Up



*Caption: The settle-up flow allows users to record payments made outside the app (cash, bank transfer) and mark debts as resolved.*

### Key Strengths

- Excellent group expense splitting with multiple split methods
- Debt simplification algorithm minimizing transfer count
- Clear visual balance summaries
- Support for multiple currencies
- Settle-up flow for recording external payments

### Key Limitations

- No personal finance management (no wallets, budgets, or personal transaction tracking)
- No AI-powered financial analysis or recommendations
- No savings goal features
- Premium features locked behind paywall

---

## C.3. Comparison and Differentiation

*Performed by: Ngô Thái Hòa & Vương Đức Gia Khiêm | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Ngô Thái Hòa & Vương Đức Gia Khiêm*

Feature Comparison Table

| Feature                          | Money Lover | Splitwise | FinSync (Ours) |
|----------------------------------|-------------|-----------|----------------|
| Personal wallet management       | ✓           | ✗         | ✓              |
| Personal transaction tracking    | ✓           | ✗         | ✓              |
| Budget planning & alerts         | ✓           | ✗         | ✓              |
| Financial reports & charts       | ✓           | Limited   | ✓              |
| Savings goals                    | ✓           | ✗         | ✓              |
| Group expense management         | ✗           | ✓         | ✓              |
| Smart debt splitting             | ✗           | ✓         | ✓              |
| Debt simplification algorithm    | ✗           | ✓         | ✓              |
| Payment reminders                | ✗           | ✓         | ✓              |
| AI financial advisor             | ✗           | ✗         | ✓              |
| Admin panel                      | ✗           | ✗         | ✓              |
| Unified personal + group finance | ✗           | ✗         | ✓              |

What Our App Will Do Differently or Better

- Unified Ecosystem:** FinSync bridges the gap between personal finance management and group expense splitting — users no longer need two separate apps.
- AI-Powered Financial Advisor:** Neither Money Lover nor Splitwise offers AI-driven spending analysis. Our app will proactively analyze spending patterns and suggest optimized budgets for the next month.
- Seamless Personal-Group Integration:** When a group expense is recorded, the system automatically notifies all members involved and tracks debts alongside personal finances, providing a holistic financial overview.
- Localized for Vietnamese Users:** Designed with Vietnamese users in mind (VND currency, local spending categories, Vietnamese UI translations), while maintaining international currency support.

UI/UX Patterns We Plan to Adopt

- From Money Lover:** Clean dashboard layout, icon-based category selection, wallet balance aggregation view, interactive report charts (pie/bar).
- From Splitwise:** Group balance color coding (green/red), simplified debt summary view, settle-up flow for external payments, flexible expense splitting UI (equal/percentage/exact).

D - Team Contract

Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long

D.1. Team Roles and Responsibilities

Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long

All members participate as Full-stack Engineers throughout the entire project lifecycle. The roles below define the primary lead responsibilities for each member:

| # | Student ID | Full Name           | Email                     | Main Role                       | Primary Responsibilities (Lead)                                                                   |
|---|------------|---------------------|---------------------------|---------------------------------|---------------------------------------------------------------------------------------------------|
| 1 | 24120087   | Phạm Đình Tiểu Long | phamlongkh2006@gmail.com  | Project Manager / Group Leader  | Manage project progress, coordinate tasks, facilitate Sprint meetings, manage Jira board          |
| 2 | 24120038   | Nguyễn Phú Đạt      | nguyennphuudatt@gmail.com | UI/UX Designer & Frontend Lead  | Lead UI/UX design (wireframes/mockups), ensure interface consistency, lead client app development |
| 3 | 24120403   | Nguyễn Lê Đức Nhật  | nldnhat182006@gmail.com   | Backend Lead                    | Lead backend architecture design, build Spring Boot APIs, design database schema                  |
| 4 | 24120342   | Vương Khắc Gia Kiên | vuongkiemvl10@gmail.com   | QA Lead & DevOps                | Lead testing strategy, author test cases, manage CI/CD pipeline, system deployment                |
| 5 | 24120051   | Ngô Thái Hòa        | ngothaihoa235@gmail.com   | AI Feature Lead & Documentation | Lead AI feature implementation (Gemini/OpenAI integration), manage technical documentation        |

**Note:** All members participate actively in peer reviews, and all technical and documentation submissions are subject to review by the respective leads and the Group Leader.

D.2. Communication Plan

Performed by: Phạm Đình Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Đình Tiểu Long

Communication Tools

| Tool               | Purpose                                          | Frequency          |
|--------------------|--------------------------------------------------|--------------------|
| Discord / Zalo     | Daily discussions, quick questions, file sharing | Daily              |
| Google Meet / Zoom | Sprint meetings, Sprint review, Sprint planning  | 4 times per Sprint |
| Jira               | Task management, progress tracking               | Checked daily      |
| GitHub             | Code reviews, pull requests, documentation       | As needed          |
| Email              | Formal communications, submissions               | As needed          |

Meeting Schedule

- **Sprint Planning:** First day of each Sprint (1 meeting)
- **Scrum Meetings:** Twice during each Sprint, spread evenly (2 meetings)
- **Sprint Review:** Last day of each Sprint (1 meeting)

Communication Protocols

- All members must respond to messages within **12 hours** during weekdays.
- All members must respond to messages within **24 hours** during weekends.
- Decisions are documented in meeting notes and shared via the communication channel.

- If a member cannot attend a meeting, they must notify at least **24 hours in advance** and provide written updates.

D.3. Work Schedule and Deadlines

Performed by: [Phạm Định Tiểu Long] | Reviewed by: [Nguyễn Lê Đức Nhật] | Edited by: [Phạm Định Tiểu Long]

Project Milestones

| Milestone | Deadline   | Description                                                    |
|-----------|------------|----------------------------------------------------------------|
| PA1       | Week 2     | Project proposal, app survey, team contract, tool setup        |
| PA2       | Week 4-5   | Requirements specification, use case diagrams, UI design       |
| PA3       | Week 7-8   | Software architecture, detailed design, initial implementation |
| PA4       | Week 10-11 | Implementation, testing, final delivery                        |

Availability and Work Sessions

- Members are available for meetings: **Sunday 7-9 PM**
- Each member commits to at least **1 hours per week** on project work.
- Tasks are assigned during Sprint Planning and tracked via Jira.

Contingency Plans

- If a member cannot meet a deadline, they must notify the team **at least 1 days before** the deadline.
- The team will redistribute the workload if a member faces unexpected difficulties.
- If a member drops the course, remaining members will redistribute tasks during the next Sprint Planning.

D.4. Code and Documentation Standards

Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long

Coding Conventions

- **Kotlin (Android Client):** Follow [Kotlin Documentation](#) style guide. Use **lowerCamelCase** for variables/functions, and **UpperCamelCase** for classes and composables.
- **Java (Spring Boot):** Follow standard naming conventions for the respective backend language. Use meaningful class, package, and method names.
- **Python (FastAPI / AI Microservice):** Follow [PEP 8](#) style guide. Use **snake\_case** for variables, functions, and filenames, and **UpperCamelCase** for classes.
- **General:** All code must include meaningful comments for complex logic (such as Notification Regex parsing algorithms or AI data formatting). Variable and function names must be descriptive and in English.

Code Review Process

- All code changes must be submitted via **pull requests (PRs)** on GitHub.
- Every PR must be reviewed by **at least 1 other team member** before merging.
- PR descriptions must clearly explain what was changed and why.
- The reviewer must test the code locally before approving.

Documentation Standards

- All documents are written in **English** using **Markdown format**.
- Diagrams are created using **Mermaid syntax** when possible (such as system architectures and data flows).
- Each section must include the **Performed by | Reviewed by | Edited by** attribution.

- Documentation is version-controlled in the `/docs` folder of the Git repository.

## D.5. Accountability and Performance

*Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long*

### Contribution Measurement

- Work is tracked via Jira task completion and GitHub commit history.
- Each Sprint Review includes a peer assessment where members rate each other's contributions.
- The Project Manager maintains a contribution log updated weekly.

### Handling Underperformance

1. **First occurrence:** Private discussion with the member and the Project Manager to identify issues and provide support.
2. **Second occurrence:** Team meeting to address the issue collectively and reassign tasks if needed.
3. **Third occurrence:** The issue is escalated to the TA or instructor for intervention.

### Consequences

- Consistently underperforming members may receive a reduced individual grade as per the team's peer evaluation.
- Failure to complete assigned tasks without valid reason will be documented and reported to the instructor.

## D.6. Decision-Making Process

*Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long*

- **Technical decisions** (architecture, tools, libraries) are made by **majority vote** after discussion.
- **Design decisions** (UI/UX, user flows) are led by the UI/UX Designer but require team consensus.
- In case of a **tie or deadlock**, the Project Manager / Group Leader has the final say.
- All major decisions are documented in meeting notes with the reasoning recorded.

## D.7. Conflict Resolution

*Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long*

### Resolution Framework

1. **Step 1 - Direct Discussion:** The involved parties discuss the issue privately and try to reach a resolution.
2. **Step 2 - Team Mediation:** If Step 1 fails, the issue is brought to the full team for mediation during a meeting.
3. **Step 3 - Leader Arbitration:** If the team cannot resolve it, the Project Manager makes the final decision.
4. **Step 4 - Escalation:** If the conflict persists or involves serious issues (e.g., academic dishonesty, harassment), it is escalated to the TA or course instructor.

### Ground Rules

- Disagreements must be professional and focused on the issue, not personal.
- All team members must respect each other's opinions and contributions.
- No unilateral decisions that affect the entire team.

## D.8. Review and Update Process

*Performed by: Phạm Định Tiểu Long | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Phạm Định Tiểu Long*

- The team contract is reviewed at the **end of each Sprint** during the Sprint Review meeting.
- Any member can propose amendments to the contract at any time.
- Amendments require a **majority vote** to be adopted.
- All changes to the contract are version-controlled in the Git repository with a clear commit message.

## E - Development Tools and Process Setup

*Performed by: Nguyễn Phú Đạt | Reviewed by: Nguyễn Lê Đức Nhật | Edited by: Nguyễn Phú Đạt*

### E.1. Development Process

- Process methodology: Scrum.
- Process:
  - 1 sprint is fixed for 2-3 weeks, corresponding with each PA.
  - There are 4 meetings for each sprint:
    - 1 sprint planning meeting: plan for the next sprint
      - This meeting needs to be held at the end of the current sprint and before the next sprint.
      - Identify and prioritize tasks, user-story or a part of a use-case for the next sprint
      - Assign tasks to team
    - 2 weekly scrum meetings
      - Each member answers 3 questions:
        - What have I done since last week?
        - What will I do until next week?
        - What issues / problems / obstacles do I have?
      - Use a single document for all scrum meetings.
    - 1 sprint review (retrospective) meeting:
      - This meeting is held at the end of each sprint to review and evaluate the results of the current sprint.
      - In this meeting, the team needs to discuss and record the following:
        - What went well.
        - What went wrong.
        - What problems and what caused the problems.
        - What can be done differently in the next sprint to improve
        - What lessons we could learn.

#### Weekly Scrum Meeting Minutes

===== 27/09, Sprint 1 (Sprint Planning) =====

Team members present:

- Phạm Định Tiểu Long
- Nguyễn Lê Đức Nhật
- Ngô Thái Hòa
- Vương Đắc Gia Khiêm
- Nguyễn Phú Đạt

Team members absent:

- None

Agenda & Planning:



- Deconstruct PA1 project assignment specification and define deliverables for Sprint 1.
- Establish workload distribution and assign specific roles to team members:
  - **Phạm Đình Tiểu Long (Group Leader):** Coordinate team tasks, initialize GitHub repository skeleton (repository creation only), author Section A (Group Registration) and Section D (Team Contract).
  - **Nguyễn Lê Đức Nhật (Technical Lead):** Author Section B (Project Proposal: Introduction, Target Users, 10 Functional Groups), set up project structure after repository creation, configure Spring Boot dependencies, author Dockerfiles (Dockerfile, docker-compose.yml), and review all members' report sections.
  - **Ngô Thái Hòa:** Author Section B.4 (AI Feature specification and Google Gemini 1.5 API workflow), and conduct comprehensive survey of Money Lover application (5 core screens, analysis, strengths, limitations) for Section C.1.
  - **Vương Đắc Gia Khiêm:** Conduct comprehensive survey of Splitwise application (5 core screens, analysis, strengths, limitations) and write Section C.2.
  - **Nguyễn Phú Đạt:** Set up development tools and process (Section E: communication channels, Jira Scrum project and board, GitHub workflow, AI tools, Weekly Reports).
- Formulate Sprint 1 backlog tasks and agree on milestone delivery deadlines.

Summary of the meeting:

- All members aligned on scope, technology stack, and individual task ownership. Sprint 1 officially kicked off.

===== 01/10, Sprint 1 (Weekly Scrum 1) =====

Team members present:

- Phạm Đình Tiểu Long
- Nguyễn Lê Đức Nhật
- Ngô Thái Hòa
- Vương Đắc Gia Khiêm
- Nguyễn Phú Đạt

Team members absent:

- None

Status reports:

- Phạm Đình Tiểu Long
  - Completed tasks
    - Delegated tasks and initial work packets to all team members
    - Initialized empty GitHub repository "pdtLong2929/FinSync" (skeleton structure only)
    - Authored Section A (Group Registration) and initial draft of Section D (Team Contract)
  - To-do Tasks
    - Finalize Section D (Team Contract: Roles, Responsibilities, Ground Rules, Accountability, and Decision-Making)
    - Coordinate sprint activities and track submission milestones
  - Issues/Obstacles
    - None
- Nguyễn Lê Đức Nhật
  - Completed tasks
    - Set up project structure after repository initialization

- Configured Spring Boot backend dependencies in pom.xml
  - Authored draft of Section B (Introduction, Target Users, and core module descriptions)
  - Authored Dockerfile and docker-compose.yml for containerized PostgreSQL service
  - To-do Tasks
    - Finalize Section B (Proposal) with detailed scope for all 10 modules
    - Review and consolidate draft report sections submitted by team members
    - Verify and test Docker container builds and dependency configurations
  - Issues/Obstacles
    - None
- Ngô Thái Hòa
  - Completed tasks
    - Surveyed Money Lover application (captured 5 key screens: Home, Transaction Entry, Budgeting, Reports, Wallets)
    - Drafted overview, strengths, and limitations for Money Lover (Section C.1)
    - Authored initial draft of Section B.4 (AI Financial Advisor specification)
  - To-do Tasks
    - Finalize Section B.4 (AI Financial Advisor specification and Gemini 1.5 API workflow)
    - Finalize Section C.1 (Money Lover survey, analysis, and key takeaways)
    - Contribute Money Lover feature comparison data to Section C.3
  - Issues/Obstacles
    - None
- Vương Đức Gia Khiêm
  - Completed tasks
    - Surveyed Splitwise application (captured 5 key screens: Group Dashboard, Add Expense, Balance Sheet, Settle Up, Activity Log)
    - Drafted overview, strengths, and limitations for Splitwise (Section C.2)
  - To-do Tasks
    - Finalize Section C.2 (Splitwise survey, analysis, and key takeaways)
    - Contribute Splitwise feature comparison data to Section C.3
  - Issues/Obstacles
    - None
- Nguyễn Phú Đạt
  - Completed tasks
    - Set up Zalo group "CNPM-2026 Nhóm 4" and Google Meet links for team communication (Section E.2)
    - Created initial Jira Scrum project "CNPM-2026 - Group 4"
  - To-do Tasks
    - Complete Section E report write-up (Jira setup, GitHub workflow, AI tools, and Weekly Reports)
    - Reconfigure Jira board and standardize Weekly Scrum meeting records according to course instructions
  - Issues/Obstacles
    - Jira project and board configuration did not yet meet standard requirements, and encountered difficulties properly formatting the Weekly Report structure

#### Actions (if any)

- Nguyễn Lê Đức Nhật to review Section E progress and provide corrective guidance on Jira configuration and Weekly Report format before Weekly Scrum 2.

## Summary of the meeting

- Project setup, Section B drafting, and app surveys are advancing according to plan (~50% progress). Nguyễn Phú Đạt reported issues regarding Jira standards and Weekly Report format, which will be addressed with review guidance.

===== 04/10, Sprint 1 (Weekly Scrum 2) =====

### Team members present:

- Phạm Định Tiểu Long
- Nguyễn Lê Đức Nhật
- Ngô Thái Hòa
- Vương Đắc Gia Khiêm
- Nguyễn Phú Đạt

### Team members absent:

- None

### Status reports:

- Phạm Định Tiểu Long
  - Completed tasks
    - Finalized Section D (Team Contract: Roles, Responsibilities, Ground Rules, Accountability, and Decision-Making)
    - Coordinated sprint activities and tracked submission milestones
  - To-do Tasks
    - Organize and lead Sprint Review & Retrospective meeting
  - Issues/Obstacles
    - None
- Nguyễn Lê Đức Nhật
  - Completed tasks
    - Finalized Section B (Proposal) with detailed scope for all 10 modules
    - Reviewed and consolidated draft report sections submitted by team members
    - Verified and tested Docker container builds and dependency configurations
  - To-do Tasks
    - Perform final quality audit of the entire consolidated report.md against PA1 specification
  - Issues/Obstacles
    - None
- Ngô Thái Hòa
  - Completed tasks
    - Finalized Section B.4 (AI Financial Advisor specification and Gemini 1.5 API workflow)
    - Finalized Section C.1 (Money Lover survey, analysis, and key takeaways)
    - Contributed Money Lover feature comparison data to Section C.3
  - To-do Tasks
    - Verify image links and Markdown formatting for Section C.1
  - Issues/Obstacles
    - None
- Vương Đắc Gia Khiêm
  - Completed tasks
    - Finalized Section C.2 (Splitwise survey, analysis, and key takeaways)

- Contributed Splitwise feature comparison data to Section C.3
- To-do Tasks
  - Verify image links and Markdown formatting for Section C.2
- Issues/Obstacles
  - None
- Nguyễn Phú Đạt
  - Completed tasks
    - Completed Section E report write-up (Jira setup, GitHub workflow, AI tools, and Weekly Reports)
    - Reconfigured Jira board and standardized Weekly Scrum meeting records according to course instructions
  - To-do Tasks
    - Revise Jira board configuration and refine Weekly Report documentation following team feedback
  - Issues/Obstacles
    - Jira project configuration and Weekly Report documentation have still not fully met the required course standards and require dedicated restructuring

#### Actions (if any)

- Nguyễn Lê Đức Nhật to perform a direct hands-on review to assist in finalizing Section E and resolving Jira/Weekly Report discrepancies.

#### Summary of the meeting

- Sections A, B, C, and D are complete and verified. Section E and Jira setup are identified as the primary items needing corrective revision before Sprint Review.

===== 08/10, Sprint 1 (Sprint Review & Retrospective) =====

#### Team members present:

- Phạm Định Tiểu Long
- Nguyễn Lê Đức Nhật
- Ngô Thái Hòa
- Vương Đắc Gia Khiêm
- Nguyễn Phú Đạt

#### Team members absent:

- None

#### Sprint Retrospective:

- What went well:
  - Clear task delegation and role boundaries established early by Phạm Định Tiểu Long; Ngô Thái Hòa and Vương Đắc Gia Khiêm completed thorough surveys for Money Lover and Splitwise respectively.
  - Comprehensive Project Proposal (Section B) structured and detailed by Nguyễn Lê Đức Nhật, enriched by Ngô Thái Hòa's dedicated AI Financial Advisor specification (Section B.4).
  - Strong technical foundation established by Nguyễn Lê Đức Nhật with robust Spring Boot dependency configuration, Dockerfiles, and thorough report consolidation.
  - Good team communication via Zalo and Google Meet with prompt responses.
- What went wrong:

- Section E deliverables assigned to Nguyễn Phú Đạt, specifically the Jira board setup and Weekly Report meeting minutes, did not meet standard course specifications on the initial submission.
- Additional time and effort were required by Nguyễn Lê Đức Nhật to review, restructure, and correct Section E and meeting minutes.
- What problems and what caused the problems:
  - Lack of familiarity with standard Jira Scrum board configuration rules and strict formatting guidelines outlined in WeeklyReport.md.
  - Insufficient early verification of Section E deliverables before mid-sprint.
- What can be done differently in the next sprint to improve:
  - Establish an early review checkpoint (mid-sprint) for all tooling, Jira tickets, and documentation artifacts.
  - Provide standardized templates and explicit checklist guidelines for Jira task management before launching Sprint 2.
- What lessons we could learn:
  - Clear individual ownership must be accompanied by continuous peer-review (led by the Technical Lead / Reviewer) to ensure high standards across all deliverables.
  - Complex tools like Jira and formal Weekly Reports require strict adherence to guidelines from day one.

## E.2. Communication Tools

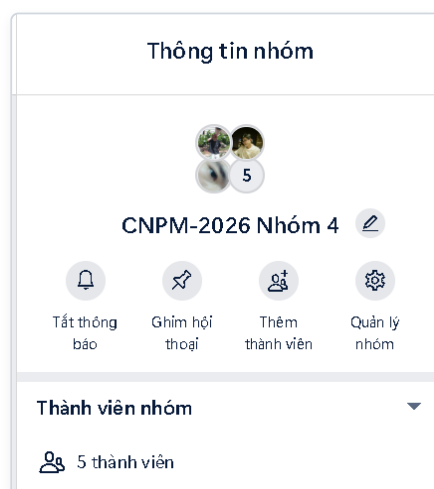
The team uses **Zalo** as the primary communication platform. The group is named “**CNPM-2026 Nhóm 4**” and currently contains five members.

Zalo is used for:

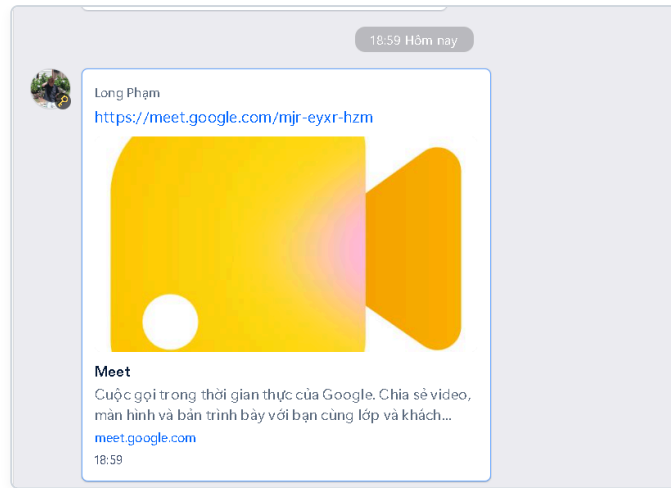
- Daily team communication.
- Discussing project tasks and requirements.
- Sharing files and documents.
- Sending meeting information and announcements.

For online meetings, the team uses **Google Meet**. Meeting links are shared through the Zalo group so that members can join discussions and project meetings.

### Evidence



Caption: Zalo communication group "CNPM-2026 Nhóm 4" with five active members.



*Caption: Google Meet virtual conference room used for team meetings and discussions.*

### E.3. Task Management with Jira

The team uses **Jira** to manage project tasks according to the Scrum process. A Scrum project named **“CNPM-2026 - Group 4”** has been created.

The Jira board uses the following workflow:

1. **To Do** – tasks that have not been started.
2. **In Progress** – tasks that are currently being worked on.
3. **Done** – completed tasks.

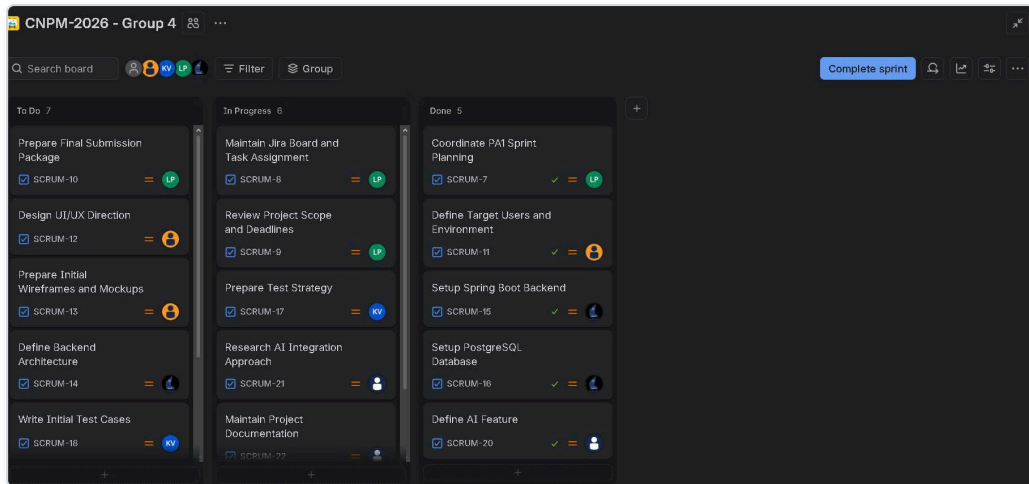
Tasks are created before the work begins and are assigned to individual team members. Each task is assigned to exactly one member so that individual responsibilities and contributions can be tracked clearly.

Examples of PA1 tasks currently managed in Jira include:

- Write Project Introduction.
- Define Target Users and Environment.
- Define Functional Groups.
- Design AI Feature.
- Research Existing Applications.
- Write Existing App Comparison.
- Write Team Contract.
- Set up Jira Project.
- Set up GitHub Repository.
- Prepare Scrum reports and Sprint Review.
- Review PA1 Documentation.
- Export Markdown documents to PDF.
- Prepare Git log evidence.
- Prepare the final PA1 submission.

The team will update task statuses throughout the sprint to reflect the actual progress of the work.

## Evidence



Caption: Jira Scrum board for project "CNPM-2026 - Group 4" showing PA1 Sprint tasks and workflow states.

### E.4. Version Control and Repository (GitHub)

The team uses **GitHub** for version control and project documentation. The project repository is named **FinSync**.

Repository URL:

<https://github.com/pdtLong2929/FinSync>

The repository currently contains the following main structure:

```
FinSync/  
├── docs/  
├── screenshots/  
├── src/  
├── .gitignore  
├── GEMINI.md  
├── README.md  
├── WeeklyReport.md  
├── docker-compose.yml  
├── pa1_2026_project_assignment_specification.md  
└── report.md
```

The repository is used to store source code, documentation, screenshots, project reports, and configuration files.

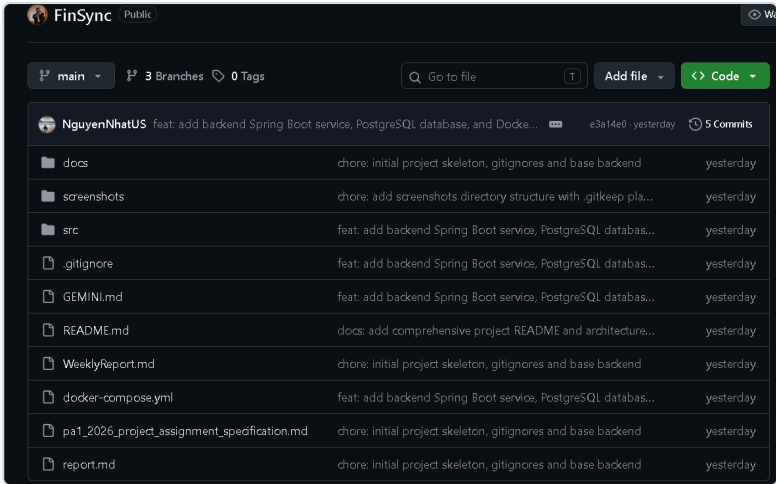
At the time of preparing this report, the repository contains **5 commits**. The repository screenshot also shows the current commit count and recent commit activity as evidence of the team's development progress and version-control activity.

Example commit activities include:

- Initial project structure and base files.
- Documentation updates.
- Backend development using Spring Boot.
- PostgreSQL database setup.
- Docker-related configuration.

- Screenshot directory preparation.

Evidence



Caption: GitHub repository showing folder structure, commit count, and recent commit activity.

Git Log Evidence

```

$ D:\Vindync\Vindync> git log --graph --all --stat
* commit 84c8da5e3ab0e0187458d0c41f159017b380 (HEAD -> main, origin/main, origin/HEAD)
Author: Nguyen Nhat <ndnhat182006@gmail.com>
Date: Sun Oct 4 19:39:45 2026 +0700

docs: initialize comprehensive project documentation, architecture design, and repository structure

GEMINI.md | 57 +++
README.md | 173 +++++
docs/README.md | 65 +++
docs/analysis-and-design/README.md | 204 ++++++
docs/analysis-and-design/api-design.md | 179 +++++
docs/analysis-and-design/architecture.md | 68 ---
docs/analysis-and-design/database-design.md | 79 ----
docs/analysis-and-design/ui-design/README.md | 91 +---
docs/management/README.md | 177 ++++++
docs/management/meeting-notes/README.md | 66 ++
docs/management/weekly-reports/README.md | 82 +---
docs/requirements/README.md | 151 ++++++
docs/requirements/functional-requirements.md | 185 +-----
docs/requirements/glossary.md | 24 --
.../non-functional-requirements.md | 35 --
docs/requirements/use-cases/README.md | 89 +---
docs/requirements/vision.md | 47 --
docs/test/README.md | 146 ++++++
report.md | 415 ++++++-----
screenshots/README.md | 14 -
screenshots/app/.gitkeep | 1 -
screenshots/app/money-lover-budget.png | Bin 0 -> 109398 bytes
screenshots/app/money-lover-dashboard.png | Bin 0 -> 124632 bytes
screenshots/app/money-lover-reports.png | Bin 0 -> 114551 bytes
screenshots/app/money-lover-transaction.png | Bin 0 -> 92634 bytes
screenshots/app/money-lover-wallets.png | Bin 0 -> 88620 bytes

* commit be1f54ef1870c7e2b1e1078776e26f34eab68ceb (origin/C_Splitwise)
Author: Khien Vuong <vuongkhien110@gmail.com>
Date: Sun Oct 4 10:40:47 2026 +0700

Splitwise screenshots

screenshots/splitwise-add-expense.png | Bin 0 -> 273388 bytes
screenshots/splitwise-balance.png | Bin 0 -> 181666 bytes
screenshots/splitwise-dashboard.png | Bin 0 -> 367139 bytes
screenshots/splitwise-group.png | Bin 0 -> 447990 bytes
screenshots/splitwise-settle.png | Bin 0 -> 111372 bytes
5 files changed, 0 insertions(+), 0 deletions(-)

* commit 208cd9cc96836cb8710861cd4c08bb21f321731
Author: Khien Vuong <vuongkhien110@gmail.com>
Date: Sat Oct 3 22:43:31 2026 +0700

Splitwise screenshots

src/screenshots/splitwise-add-expense.png | Bin 0 -> 273388 bytes
src/screenshots/splitwise-balance.png | Bin 0 -> 181666 bytes
src/screenshots/splitwise-dashboard.png | Bin 0 -> 367139 bytes
src/screenshots/splitwise-group.png | Bin 0 -> 447990 bytes
src/screenshots/splitwise-settle.png | Bin 0 -> 111372 bytes
5 files changed, 0 insertions(+), 0 deletions(-)

* commit ea866842ba72f598ce75a2044f7d28c4c79d8b1c1 (origin/C_moneylover)
Author: ThaisHoaf <241200518@student.hcmus.edu.vn>
Date: Sun Oct 4 06:55:00 2026 +0700

docs: add images to illustrate for Money Lover

screenshots/money-lover-budget.png | Bin 0 -> 109398 bytes
screenshots/money-lover-dashboard.png | Bin 0 -> 124632 bytes
screenshots/money-lover-reports.png | Bin 0 -> 114551 bytes
screenshots/money-lover-transaction.png | Bin 0 -> 92634 bytes
screenshots/money-lover-wallets.png | Bin 0 -> 88620 bytes
5 files changed, 0 insertions(+), 0 deletions(-)

* commit e3a14e01ba4431dadae1861b54054453c969b44c
Author: Nguyen Nhat <ndnhat182006@gmail.com>
Date: Sat Oct 3 15:41:40 2026 +0700

feat: add backend Spring Boot service, PostgreSQL database, and Docker configuration

.gitignore | 2 +
GEMINI.md | 24 +++++
docker-compose.yml | 79 ++++++
src/backend/.env.example | 19 +---
src/backend/.gitignore | 8 ++
src/backend/Dockerfile | 32 ++++++
.../main/resources/application.properties | 36 ++++++
...sts.java => FinSyncApplicationTests.java | 4 +-
8 files changed, 201 insertions(+), 3 deletions(-)
```



```

* commit 98327ff7e68be6339ac57b0d906c24536c8ffac2
Author: Nguyen Nhat <nldnhat182006@gmail.com>
Date: Sat Oct 3 14:48:30 2026 +0700

    docs: add comprehensive project README and architecture documentation

 README.md | 252 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 252 insertions(+)
 Show Commit (ctrl + click)
* commit 971eb17f393e4022cd47d3757b69c76695bfefb04
Author: Nguyen Nhat <nldnhat182006@gmail.com>
Date: Sat Oct 3 14:46:07 2026 +0700

    chore: add screenshots directory structure with .gitkeep placeholders

 screenshots/README.md | 19 +++++
 screenshots/app/.gitkeep | 1 +
 screenshots/git/.gitkeep | 1 +
 screenshots/jira/.gitkeep | 1 +
 4 files changed, 22 insertions(+)

```

```

PS D:\Finsync\Finsync> git log --graph --all --stat
* commit 490f83e91557f449cbb378bf31bcd5aaa83a7fc
Author: Pham Dinh Tieu Long <phanlongh2006@gmail.com>
Date: Fri Oct 2 07:54:34 2026 +0700

    Initial commit

 README.md | 1 +
 1 file changed, 1 insertion(+)
(END)
docs/requirements/vision.md | 47 ++
docs/test/README.md | 38 +
docs/test/test-cases/.gitkeep | 1 +
docs/test/test-results/.gitkeep | 1 +
pai_2026_project_assignment_specification.md | 156 +++
report.md | 697 +++++
src/admin-web/.gitignore | 21 +
src/backend/.gitattributes | 2 +
src/backend/.gitignore | 38 +
.../maven-wrapper/maven-wrapper.properties | 3 +
src/backend/HdLP.md | 35 +
docs/test/test-cases/.gitkeep | 1 +
docs/test/test-results/.gitkeep | 1 +
pai_2026_project_assignment_specification.md | 156 +++
report.md | 697 +++++
src/admin-web/.gitignore | 21 +
src/backend/.gitattributes | 2 +
src/backend/.gitignore | 38 +
.../maven-wrapper/maven-wrapper.properties | 3 +
src/backend/HdLP.md | 35 +
src/backend/mvnw | 295 +++++
src/backend/mvnw.cmd | 189 +++++
src/backend/pom.xml | 155 +

```

```

PS D:\Finsync\Finsync> git log --graph --all --stat
* commit 490f83e91557f449cbb378bf31bcd5aaa83a7fc
Author: Pham Dinh Tieu Long <phanlongh2006@gmail.com>
Date: Fri Oct 2 07:54:34 2026 +0700

    Initial commit

 README.md | 1 +
 1 file changed, 1 insertion(+)
(END)
docs/requirements/vision.md | 47 ++
docs/test/README.md | 38 +
docs/test/test-cases/.gitkeep | 1 +
docs/test/test-results/.gitkeep | 1 +
pai_2026_project_assignment_specification.md | 156 +++
report.md | 697 +++++
src/admin-web/.gitignore | 21 +
src/backend/.gitattributes | 2 +
src/backend/.gitignore | 38 +
.../maven-wrapper/maven-wrapper.properties | 3 +
src/backend/HdLP.md | 35 +
src/backend/mvnw | 295 +++++
src/backend/mvnw.cmd | 189 +++++
src/backend/pom.xml | 155 +
.../java/com/finsync/FinsyncApplication.java | 11 +
.../main/resources/application.properties | 1 +
.../pai/Finsync/pai/ApplicationTests.java | 13 +
src/mobile/.gitignore | 38 +
35 files changed, 2765 insertions(+)

```

Caption: Git commit history log showing the team's incremental development activities for PA1.

## E.5. AI-Assisted Development Tools

The team uses several AI-assisted tools to support software development and self-learning, including:

- **ChatGPT**
- **Claude**
- **Gemini**
- **GitHub Copilot**
- **Codex**

These tools are used to support activities such as:

- Code generation and code completion.
- Debugging and error explanation.
- Learning unfamiliar technologies.
- Improving documentation.
- Reviewing implementation ideas.
- Assisting with software development tasks.

AI-generated suggestions are reviewed by team members before they are integrated into the project.

Evidence

All team members have registered and verified their educational and developer AI coding accounts to support project implementation as summarized below:

AI Coding Accounts Registration Evidence

| # | Team Member         | AI Coding Platform                 | Registration Status |
|---|---------------------|------------------------------------|---------------------|
| 1 | Phạm Định Tiểu Long | GitHub Copilot (Student) / ChatGPT | ✔ Active            |
| 2 | Nguyễn Lê Đức Nhật  | Codex / Gemini                     | ✔ Active            |
| 3 | Ngô Thái Hòa        | GitHub Copilot (Student) / Gemini  | ✔ Active            |
| 4 | Vương Đắc Gia Khiêm | GitHub Copilot (Student) / Codex   | ✔ Active            |
| 5 | Nguyễn Phú Đạt      | Cursor / GitHub Copilot            | ✔ Active            |

E.6. Spec Kit

The team has **not initialized Spec Kit during PA1**.

According to the PA1 requirements, Spec Kit setup and initialization will become required starting from **PA2**. The team will configure Spec Kit at the beginning of PA2 and use it to support specification-driven development from requirements to implementation.

E.7. Tool Setup Summary

| Area                | Tool / Platform                                | Current Status  |
|---------------------|------------------------------------------------|-----------------|
| Team Communication  | Zalo                                           | Set up          |
| Online Meetings     | Google Meet                                    | Set up          |
| Task Management     | Jira                                           | Set up          |
| Development Process | Scrum                                          | In use          |
| Version Control     | GitHub                                         | Set up          |
| Git History         | Git commits                                    | Available       |
| AI Assistance       | ChatGPT, Claude, Gemini, GitHub Copilot, Codex | In use          |
| Spec Kit            | Spec Kit                                       | Planned for PA2 |

The current tool setup provides the team with a structured environment for communication, task management, version control, documentation, and AI-assisted development. The team will continue updating Jira tasks, Git commits, meeting records, and supporting evidence throughout the sprint.